JCash is the Philippines' top desktop wallet and financial Superapp

Student Name
Jason Verzosa Begornia

Batch and Year
CIICC B14 2026

Instructor
Marco Yimyaem

https://github.com/JSON26F-DV/jcash

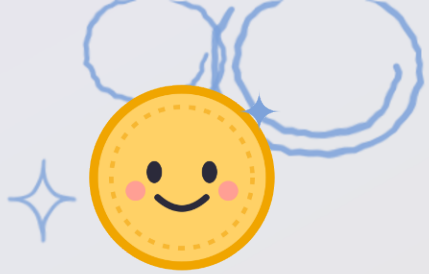
Project Description

JCash is a JavaFX desktop banking application that uses MariaDB/MySQL for user accounts, balances, and transaction records. It works like a simple e-wallet: cash in, send money, and view your transaction history.

- A JavaFX desktop GUI, not a console app.
- Clear layers: Controller, Service, DAO, Model, and Session.
- All permanent data lives in the jcash_db database.



Tools Used



JavaFX + FXML

Desktop GUI. FXML defines the layout of every screen.

MariaDB / MySQL

jcash_db stores the users, transactions, and cards.

JDBI + JDBC

Connects Java to the database and runs the queries.

Lombok

@Data generates the getters and setters of User and Transaction.

BigDecimal

Exact money values, backed by DECIMAL(15,2) in the database.

SHA-256

The PIN is hashed before it is saved, never stored as plain text.

System Overview



Screen (FXML)
layout only

→

Controller
handles UI input

→

Service
business rules

→

DAO
database queries

→

Connection
DatabaseConnection + JDBI

→

jcash_db
MariaDB / MySQL

Where does the data go?

Permanent

Balances and transaction history live in the jcash_db database.

Temporary

The User object and UserSession exist only in memory until sign out or app close.

Layout only

FXML describes how a screen looks; it never stores balances or history.

Database Schema



users Table

transactions Table

```
Sun 8 Nov - 21:04 ~
@jason /opt/lampp/bin/mysql -u root -p jcash_db -e "DESC transactions;"
Enter password:
+-------------+---------------+------+-----+---------------------+-------------------------------+
| Field       | Type          | Null | Key | Default             | Extra                         |
+-------------+---------------+------+-----+---------------------+-------------------------------+
| id          | bigint(20)    | NO   | PRI | NULL                | auto_increment                |
| sender_id   | bigint(20)    | YES  | MUL | NULL                |                               |
| receiver_id | bigint(20)    | YES  | MUL | NULL                |                               |
| amount      | decimal(15,2) | NO   |     | NULL                |                               |
| description | varchar(150)  | NO   |     | NULL                |                               |
| category    | varchar(50)   | NO   |     | Automatic           |                               |
| message     | varchar(100)  | NO   |     | NULL                |                               |
| created_at  | datetime      | NO   | MUL | current_timestamp() |                               |
| updated_at  | datetime      | NO   |     | current_timestamp() | on update current_timestamp() |
+-------------+---------------+------+-----+---------------------+-------------------------------+
```

cards Table

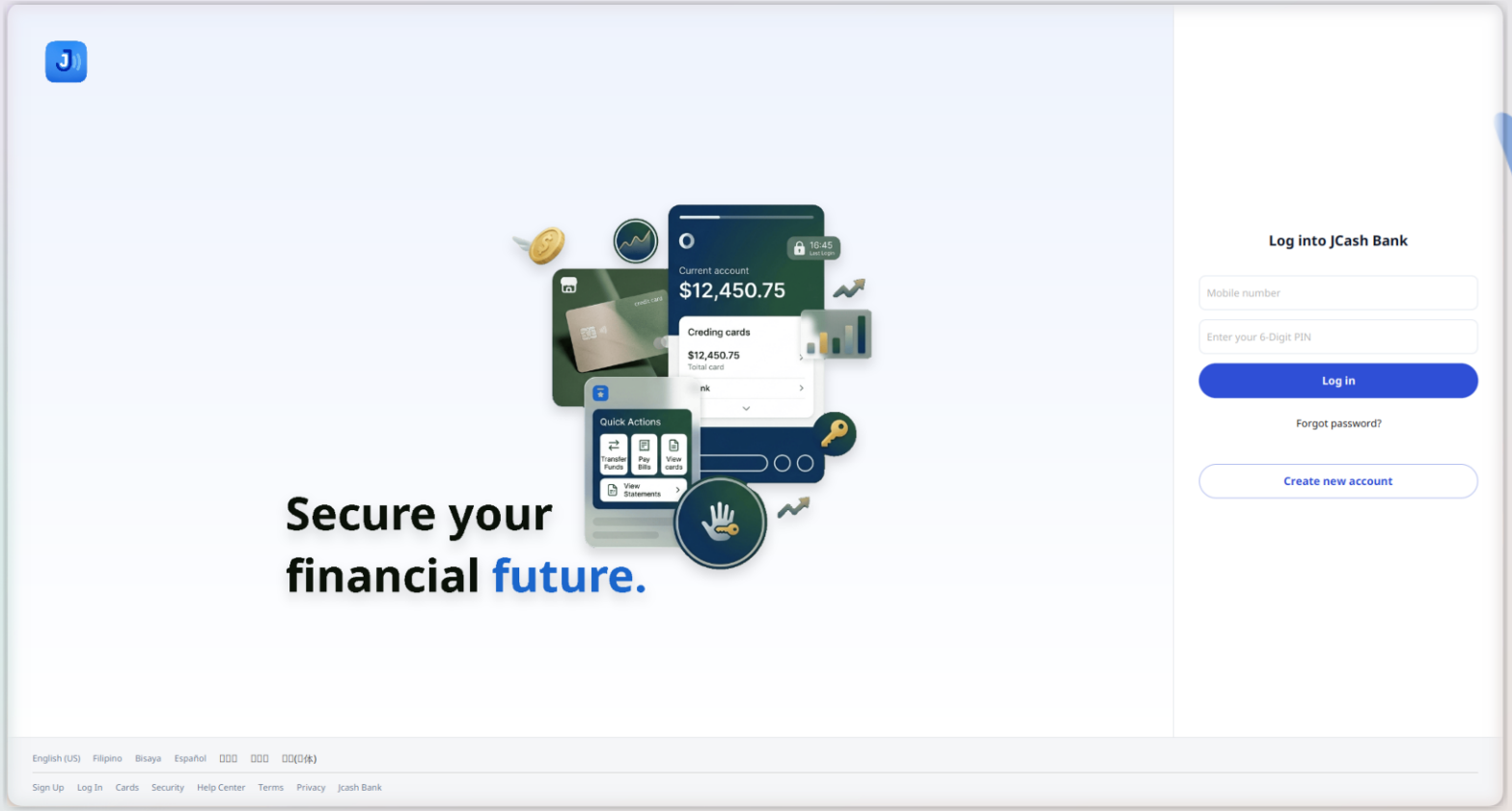
```
Sun 8 Nov - 21:05 ~
@jason /opt/lampp/bin/mysql -u root -p jcash_db -e "DESC cards;"
Enter password:
+-------------+-----------------------------------------+------+-----+---------------------+----------------+
| Field       | Type                                    | Null | Key | Default             | Extra          |
+-------------+-----------------------------------------+------+-----+---------------------+----------------+
| id          | bigint(20)                              | NO   | PRI | NULL                | auto_increment |
| user_id     | bigint(20)                              | NO   | MUL | NULL                |                |
| card_type   | enum('debit','credit','prepaid')        | NO   |     | NULL                |                |
| color       | enum('blue','graphite','onyx','emerald') | NO   |     | blue                |                |
| card_number | varchar(19)                             | NO   | UNI | NULL                |                |
| expiry_date | char(5)                                 | NO   |     | NULL                |                |
| pin_hash    | varchar(255)                            | NO   |     | NULL                |                |
| is_frozen   | tinyint(1)                              | NO   |     | 0                   |                |
| created_at  | timestamp                               | NO   |     | current_timestamp() |                |
+-------------+-----------------------------------------+------+-----+---------------------+----------------+
```

LANDING PAGE

Source Code
LandingPageController.java,
AuthService.java,

Features

- Landing page: choose to log in or create an account.
- Login uses a mobile number and PIN; too many wrong PINs trigger a lockout.



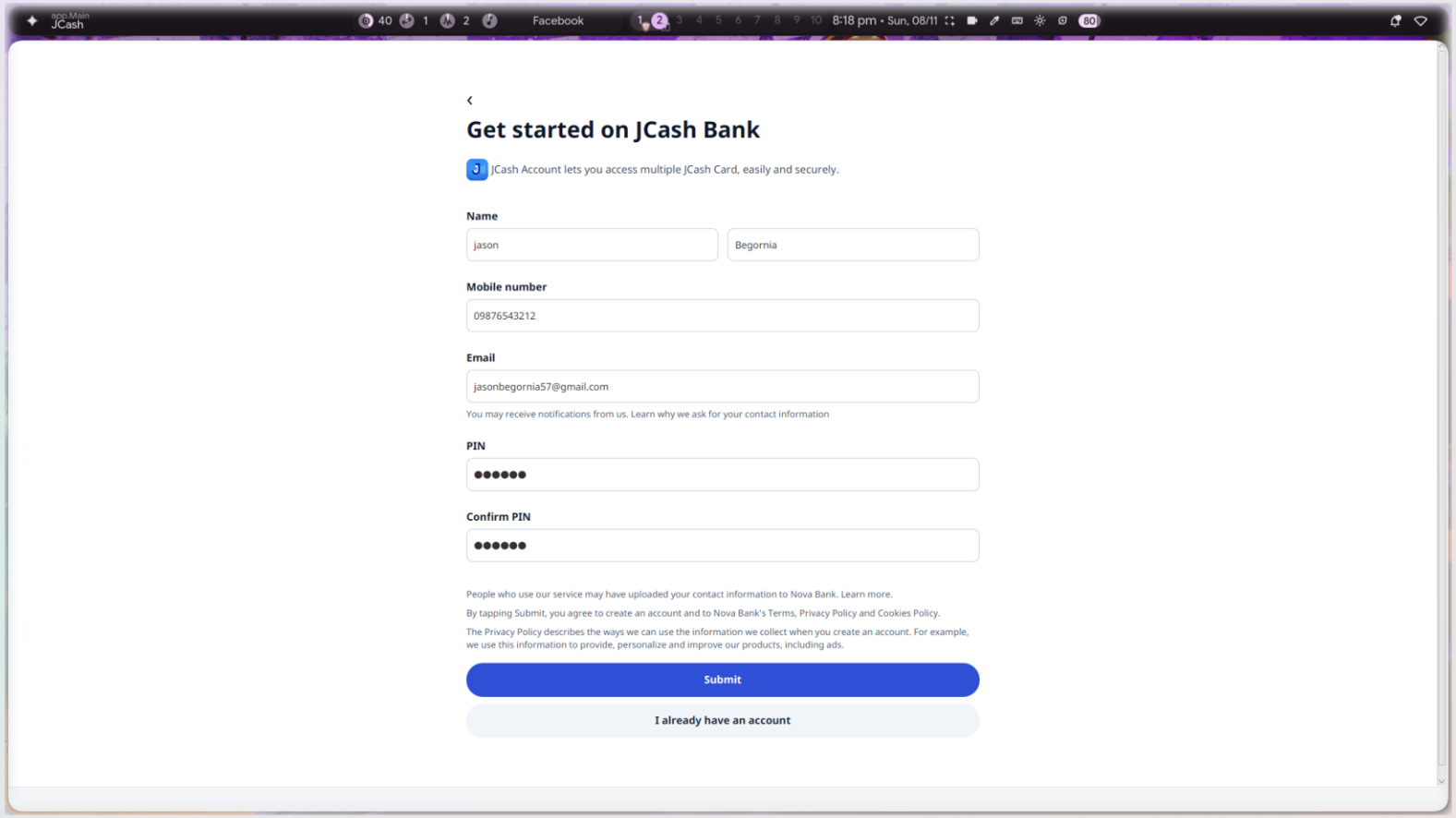
Registration page

Source Code

RegisterPageController.java, UserService.java, UserDAO.java

Features

- Registration validates the form, hashes the PIN, then saves it with UserDAO.create().

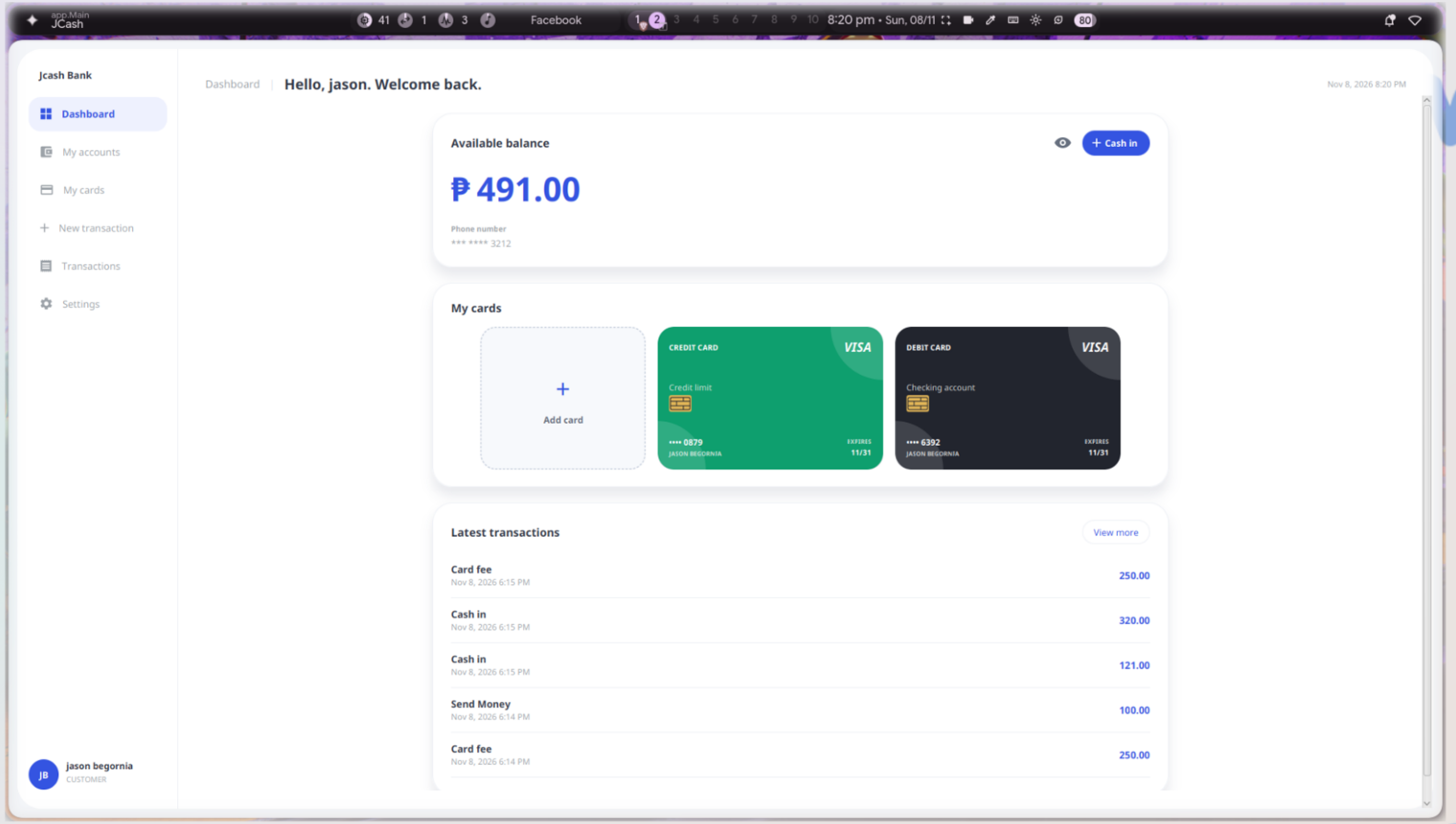


Dashboard

Source Code
DashboardController.java,
TransactionService.java,
TransactionDAO.java

Features

- The dashboard shows a welcome text, the balance, and the latest 5 transactions.
- History lists every transaction where the user is the sender or receiver, newest first.
- Selecting a row opens its details.



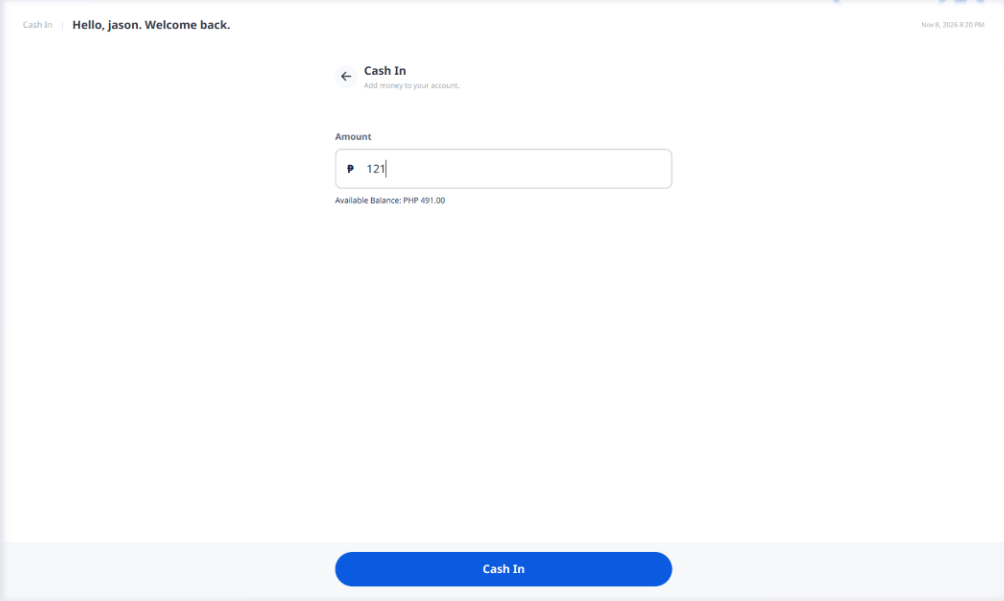
Cash In

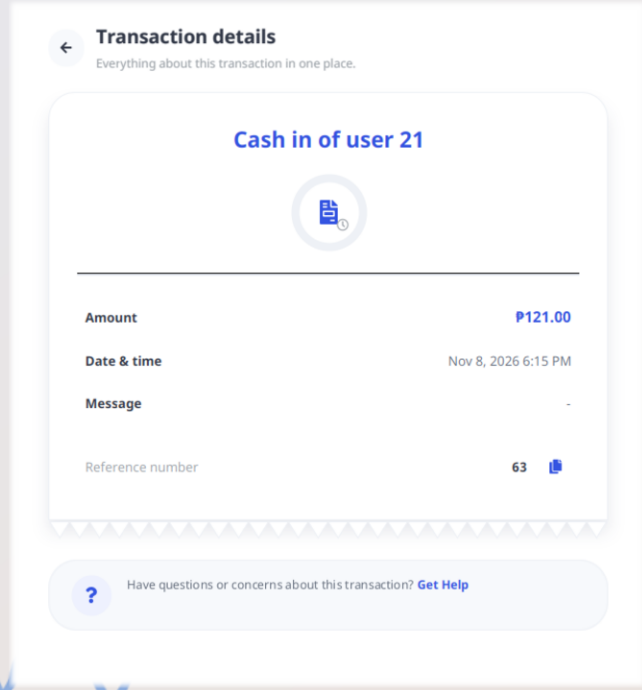
Source Code

NewTransactionController.java, TransactionService.java, TransactionDAO.java

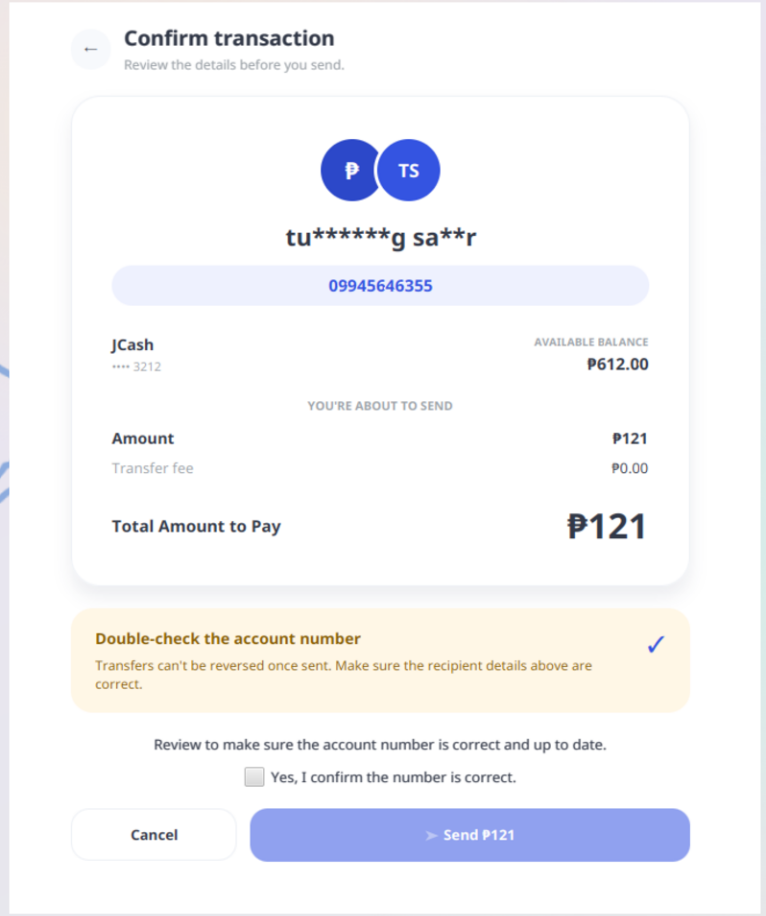
Features

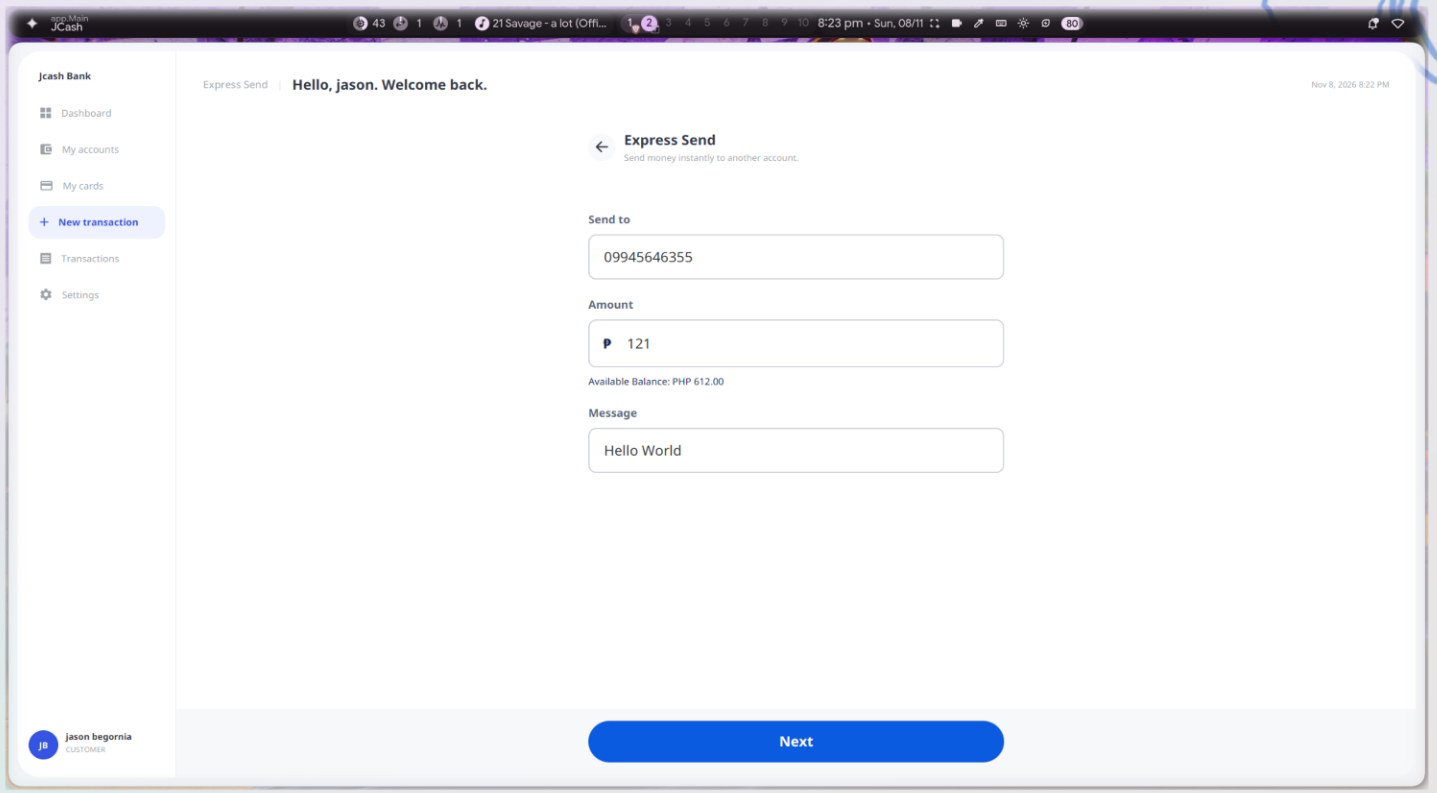
- Only positive amounts are accepted.
- TransactionDAO inserts the record and updates the balance in one database transaction.
- The balance is refreshed from the database, then a receipt is shown.





Express Send





Source Code

NewTransactionController.java, ConfirmationController.java, TransactionService.java

Features

- Enter the receiver's mobile number, the amount, and a message.
- Checks: the receiver exists, it is not your own number, and the balance is enough.
- A confirmation page appears before send() runs.

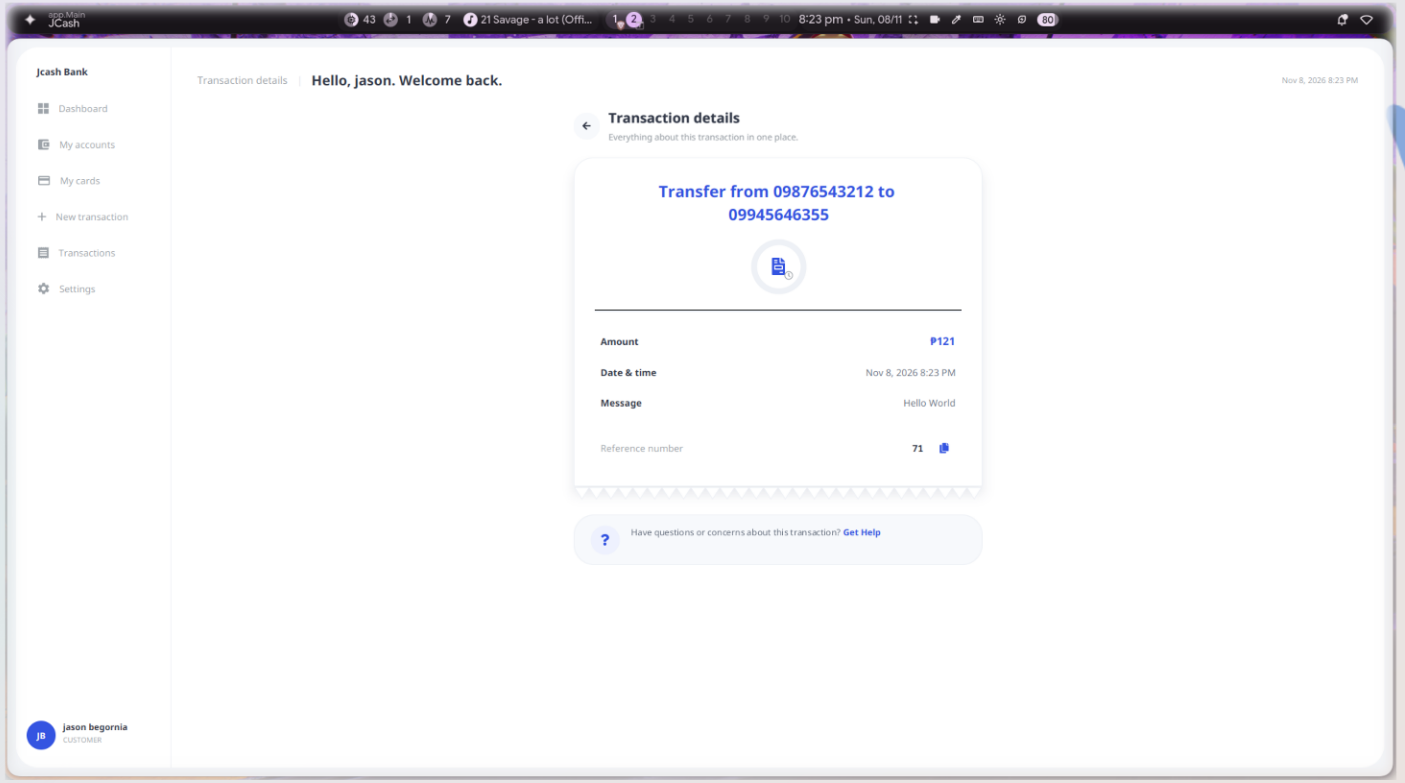
Express Send: Receipt

Source Code

TransactionService.java,
TransactionDAO.java

Features

- send() validates the user, receiver, and amount again.
- The sender gets a negative delta and the receiver a positive one, inside one database transaction.
- Any failure rolls everything back, so money is never lost or duplicated.

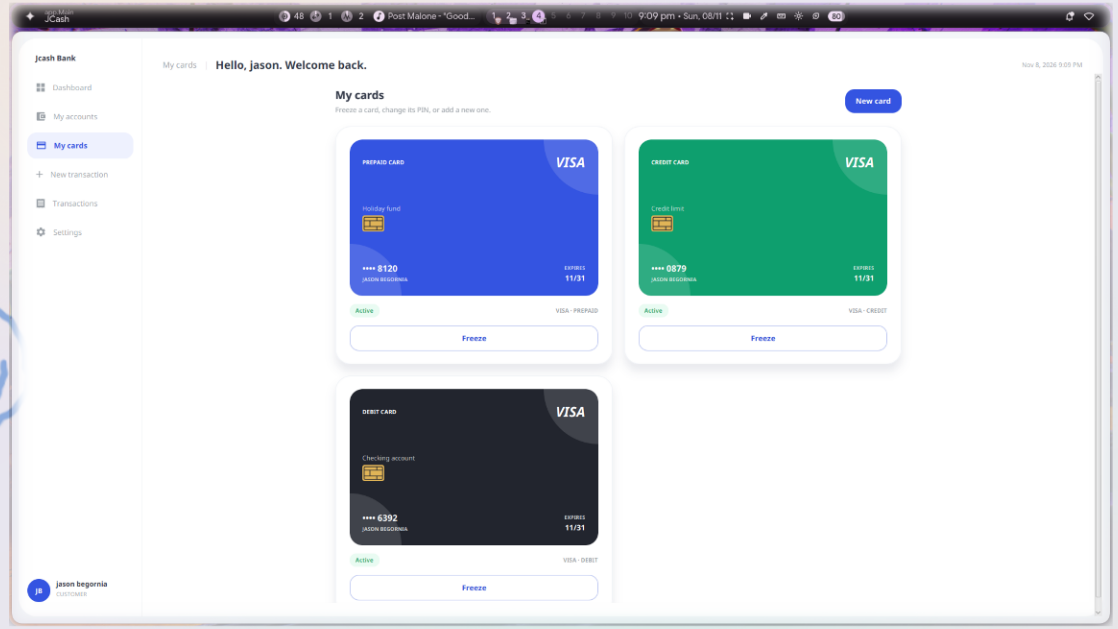


Send Receipt

Cards

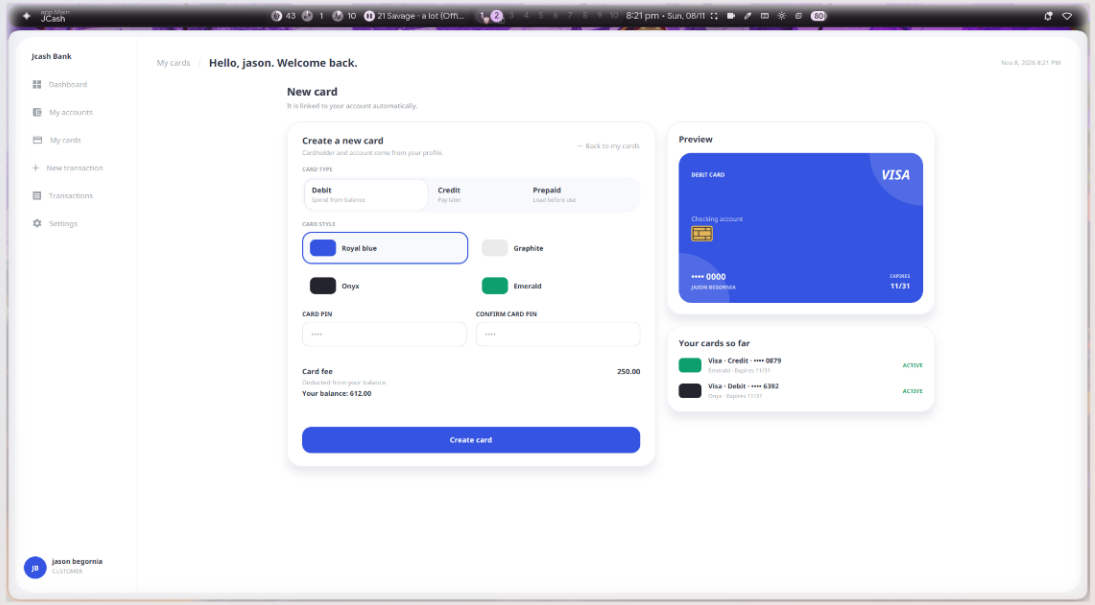
Features

- My Cards lists the cards of the signed-in user.
- Buy Card lets the user get a new card.

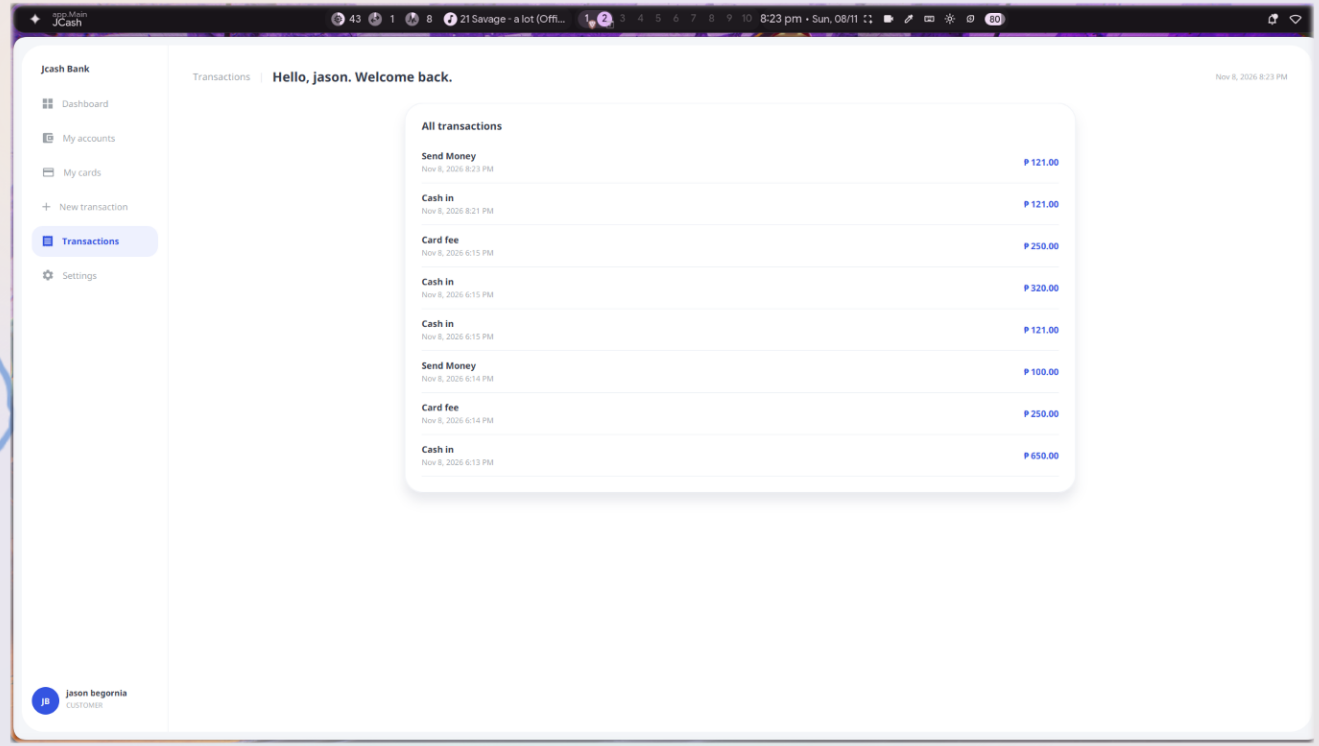


My Cards

Buy Card



Transaction History

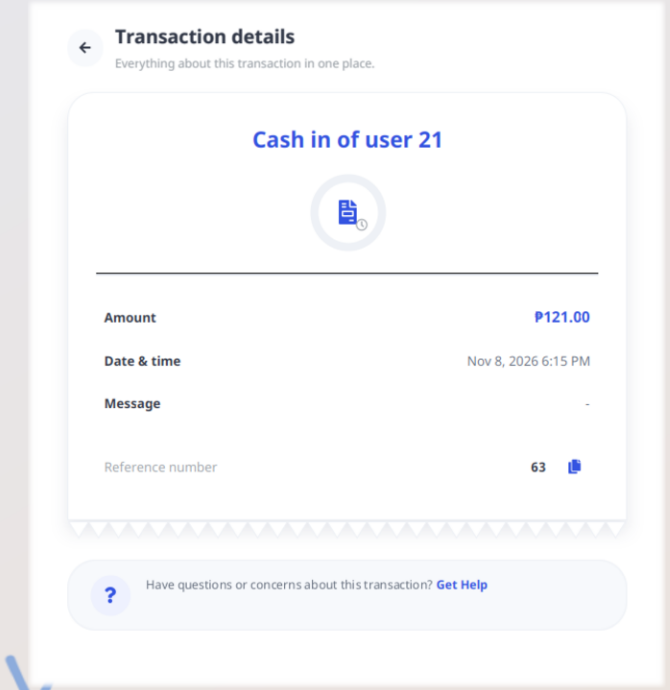


Source Code

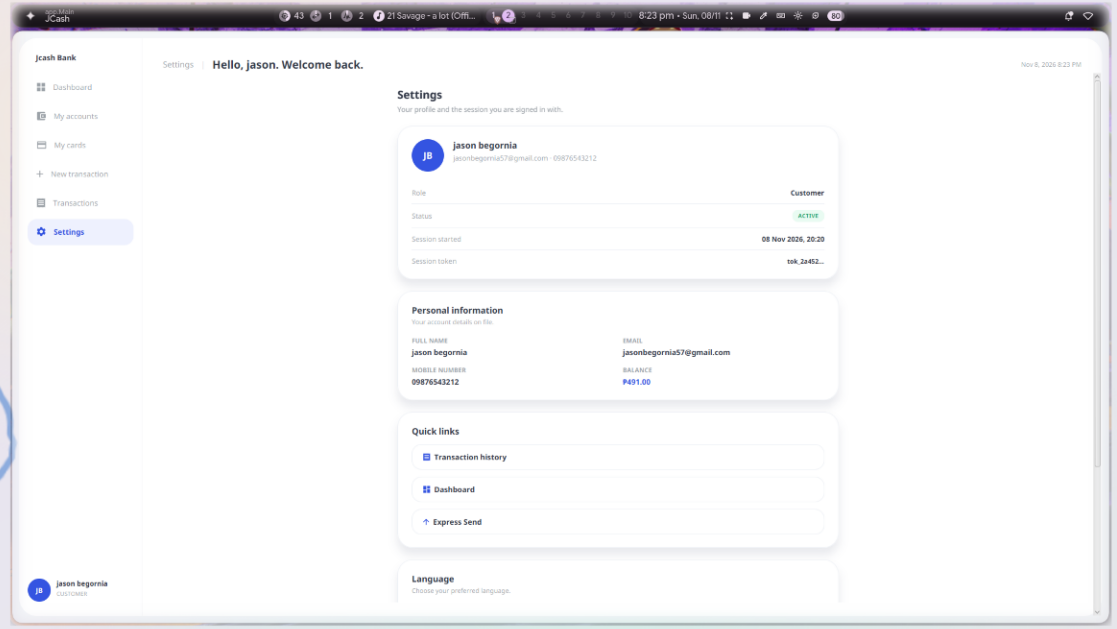
DetailsController.java, TransactionService.java, TransactionDAO.java

Features

- Show User Transactions.
- Selecting a row opens its details.



Settings



Settings

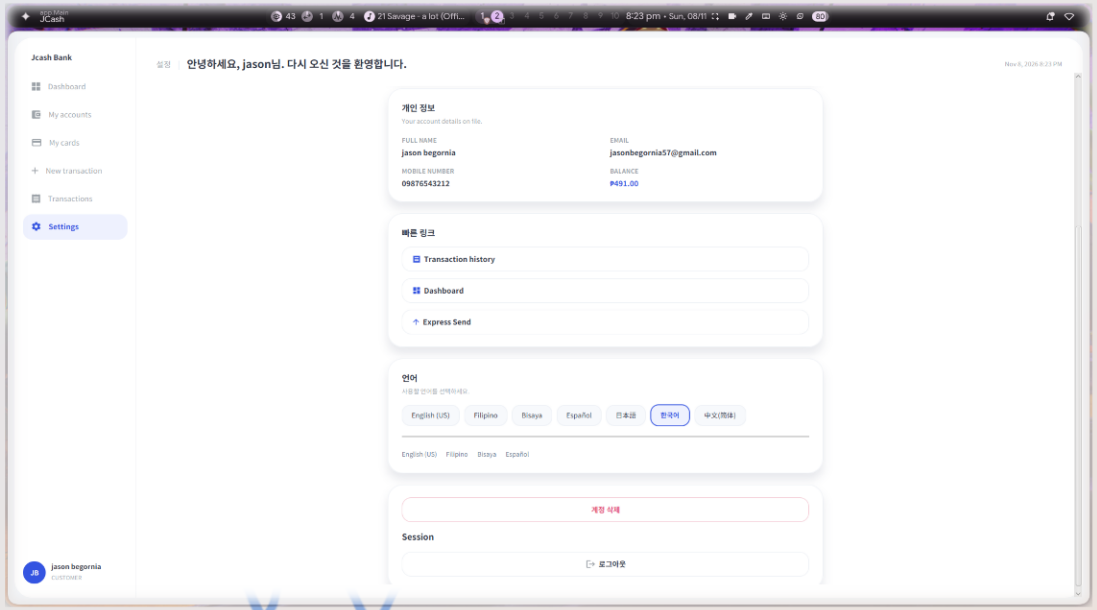
Source Code

AuthService.java, UserSession.java

Features

- Settings is the hub for account options.
- Sign Out calls AuthService.logout(), which ends the UserSession.
- The display language can be changed here.

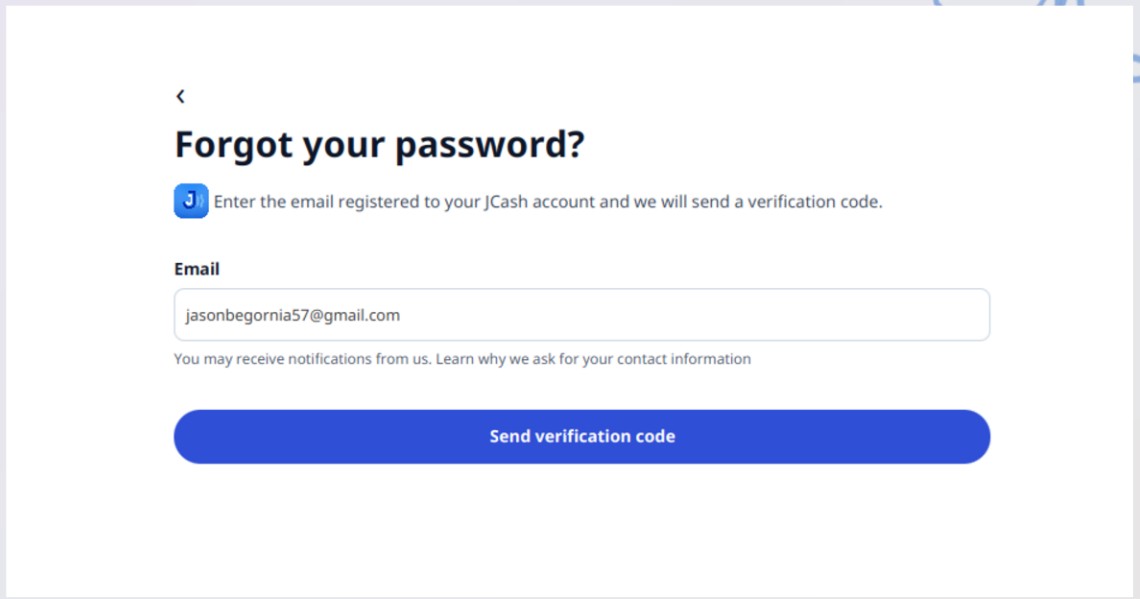
Change Language

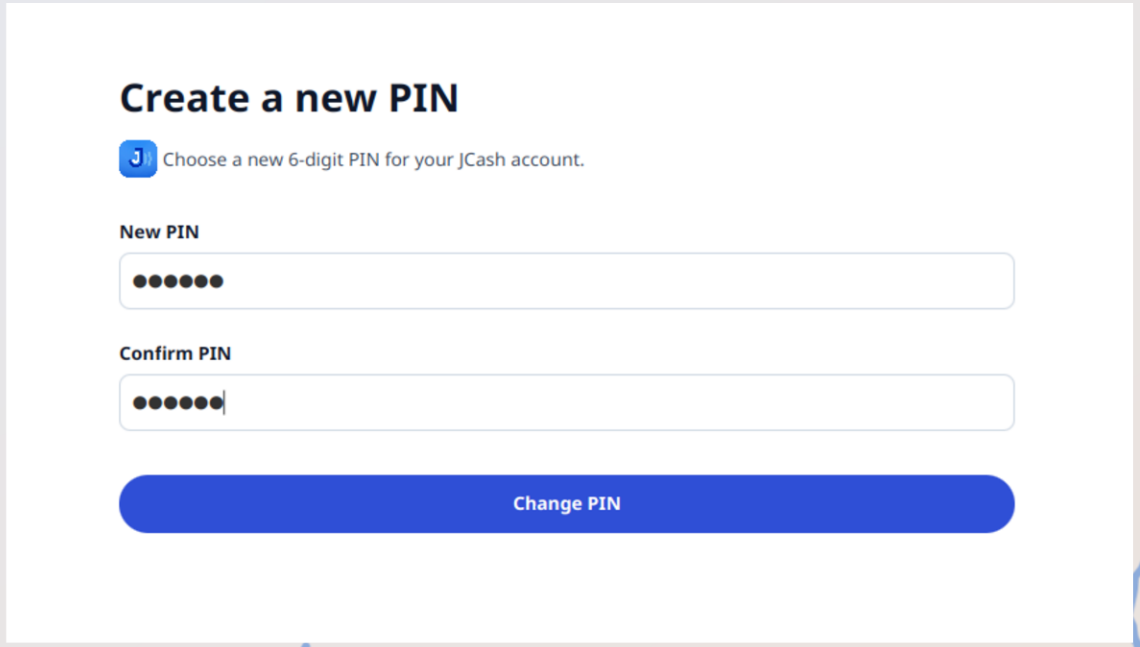


Password Management

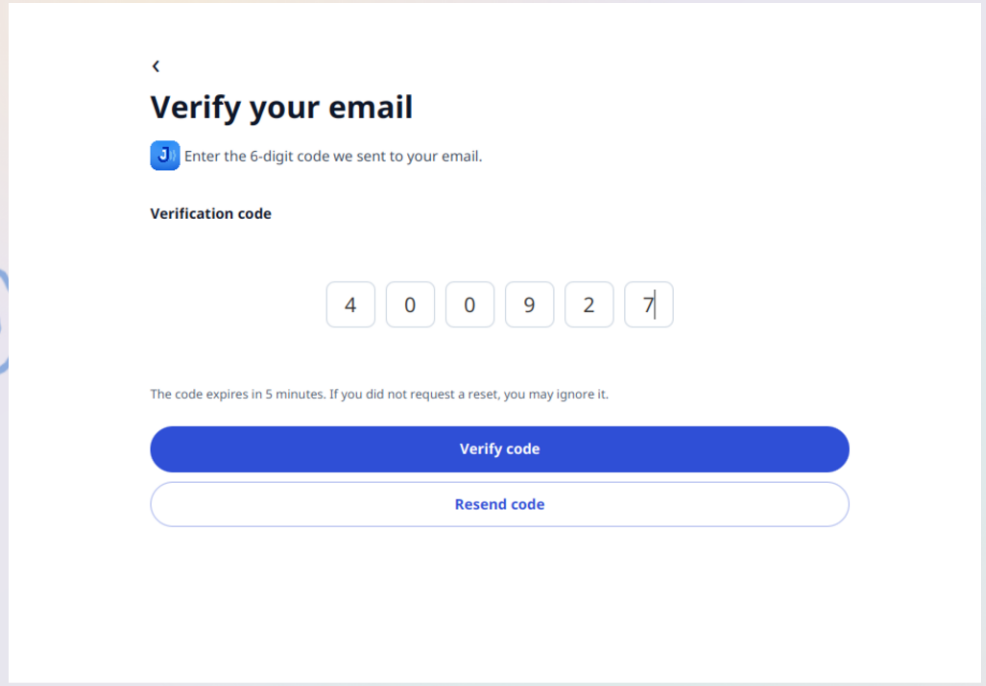
Features

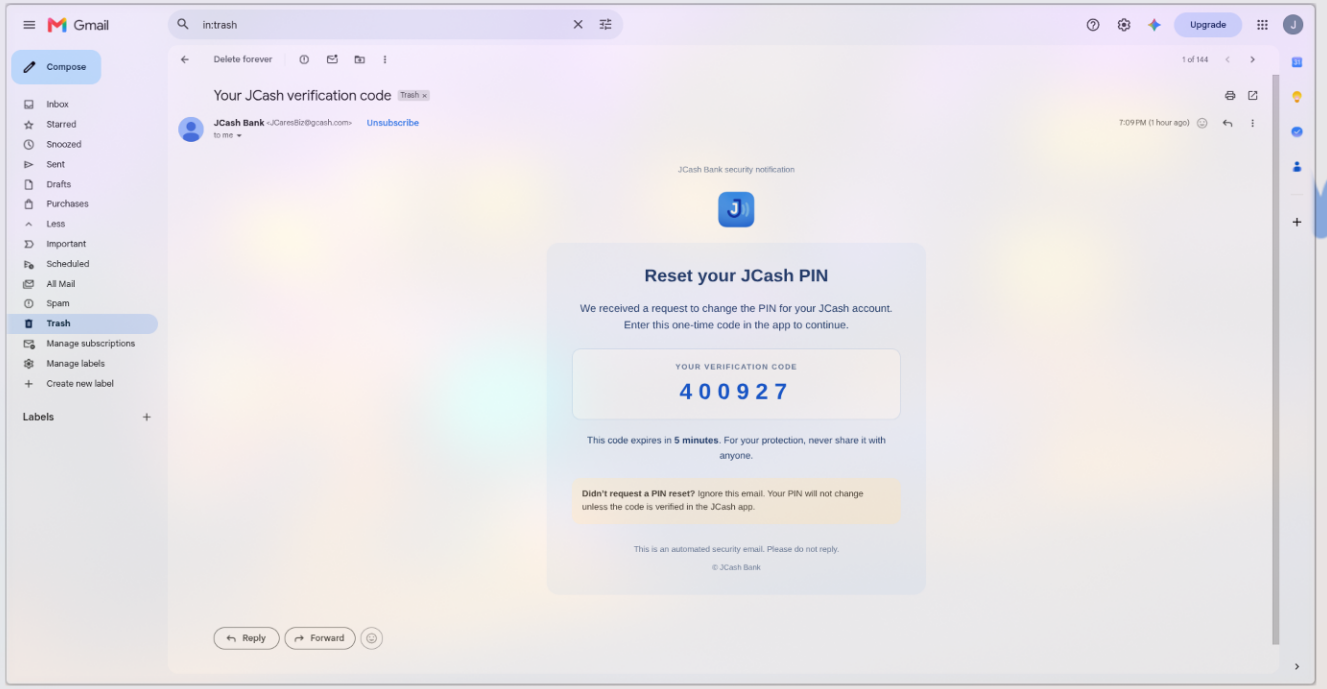
- Signed-in users can change their password.
- Forgot Password starts a reset using the account's email.





Email & OTP Verification





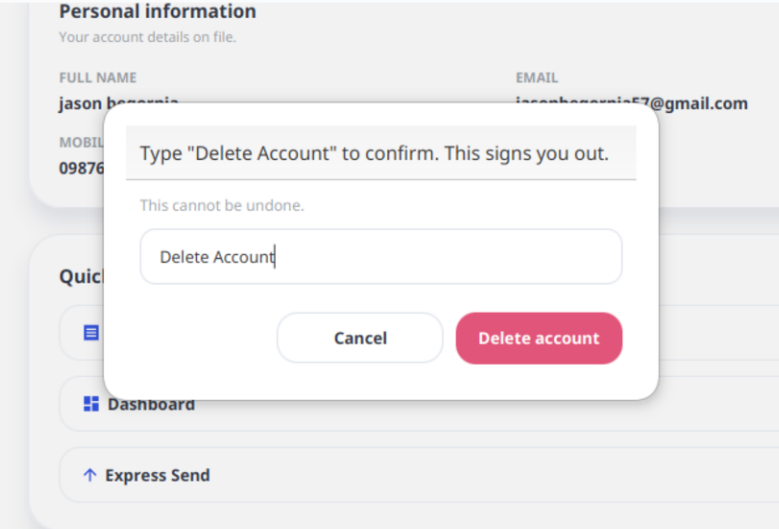
Features

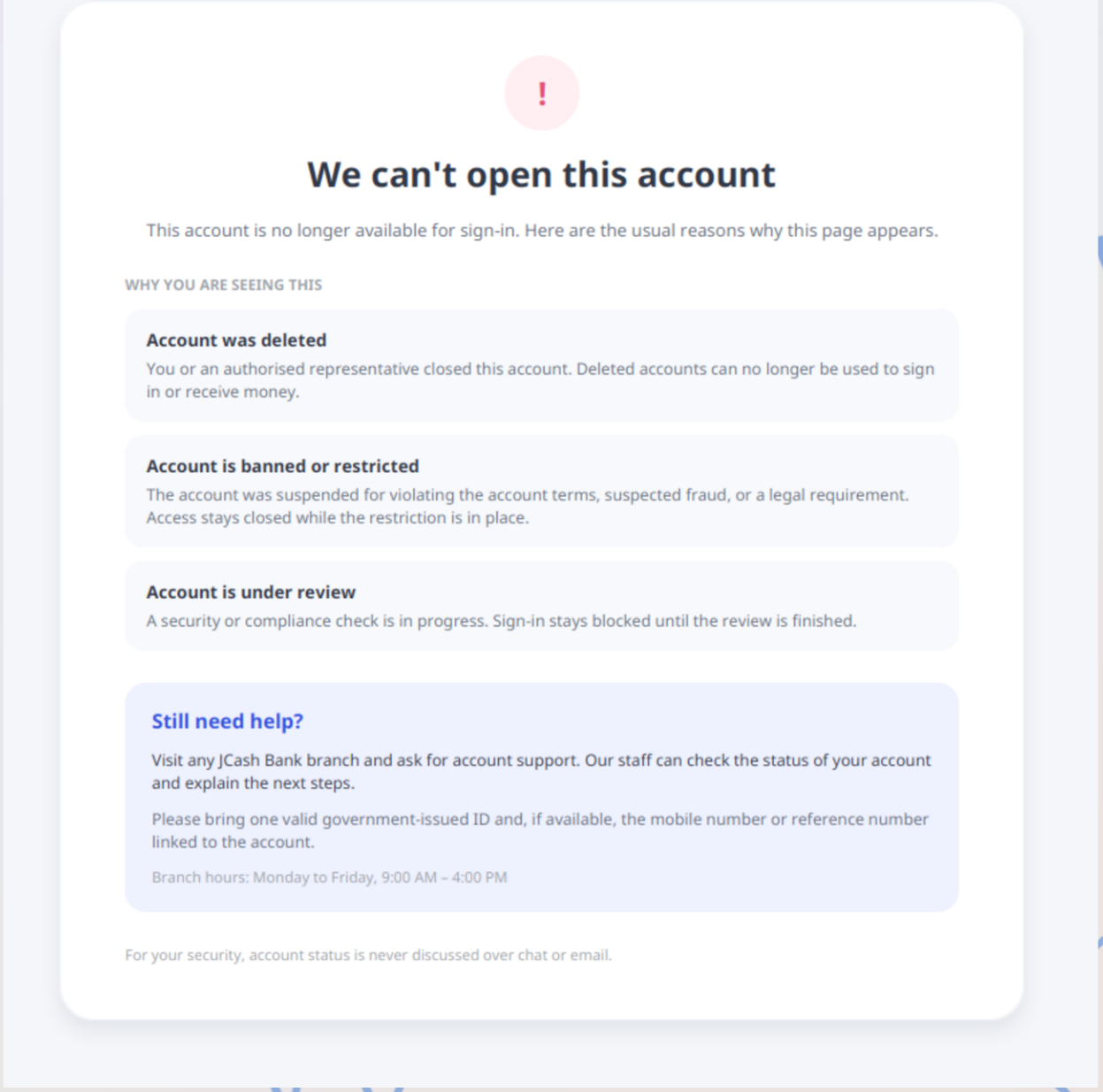
- A verification email with an OTP code is sent.
- The OTP code is checked before the reset can continue.

Delete Account

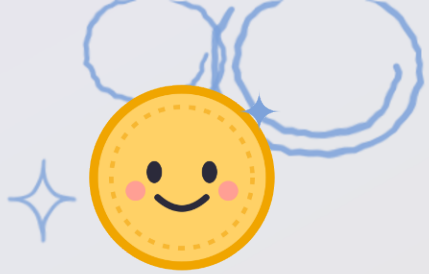
Features

- The user must confirm before the account is deleted.
- A final page confirms that the account has been removed.





Java Concepts



Encapsulation

User and Transaction hold their own fields; Lombok @Data generates getters and setters. The PIN is excluded from toString().

Abstraction

The DAO<T> interface defines create, update, delete, and selectAll. Controllers never see how rows are fetched.

Inheritance

LandingPageController extends AuthService. Composition would be a cleaner design.

Polymorphism

UserDAO and TransactionDAO both implement DAO<T> with their own behavior.

BigDecimal

Money uses BigDecimal, never double. DECIMAL(15,2) in the database; compareTo() for comparison.

Atomicity

All or nothing. A transfer saves the record, debits, and credits together, or rolls back.